

A Grapheme-Cluster Initialized Unigram Tokenizer for Bengali: Reducing Cross-Lingual Token Cost Disparity

Md. Raiyan Noor
BRAC University, Bangladesh
md.raihan.noor@g.bracu.ac.bd

Md. Shahriar Islam Bhuiyan
BRAC University, Bangladesh
shahriar.islam.bhuiyan@g.bracu.ac.bd

Abstract

Tokenizers built on an English dominant corpus tend to over-segment low resource languages such as Bengali. This leads to an inflated token count, inference cost, and effective context window for Bengali natives. This proposal is for a research plan to build a tokenizer that is Bengali-native. Rather than raw bytes, the tokenizer will initialize its vocabulary from grapheme clusters (aksharas). The tokenizer will use a unigram language model rather than byte pair encoding. Because segmentation unit and modeling objective are independent design choices, the proposed work isolates their individual and combined effects through a controlled ablation, and benchmarks the result against existing Bengali and general-purpose tokenizers using fertility, characters-per-token, parity relative to English, and Single Token Retention Rate. This document presents the motivation, background, literature review, research questions, methodology, dataset, implementation plan, evaluation plan, and timeline for the project.

1 Introduction and Motivation

Tokenization is the first computational step in processing text with a language model, converting raw text into discrete units that the model consumes. The design of a tokenizer directly affects how many tokens are needed to represent a given piece of text, which in turn affects inference cost, latency, and the effective size of the usable context window.

Tokenizers built primarily for English systematically over-segment non-Latin scripts. Bengali is affected in particular due to its multi-byte UTF-8 encoding, its extensive use of conjunct consonants formed through virama combination, and its dependent vowel signs, none of which are well represented in vocabularies trained mainly on English or other Latin-script text. As a result, Bengali text can require several times as many tokens as equivalent English content for the same meaning.

Petrov et al. [1] documented this disparity broadly across languages, reporting gaps of up to fifteen times between languages for content of equivalent meaning. More than engineering inefficiency, this is also a fairness and access problem. A higher token cost for a language means a higher monetary cost and also reduced practical usable context window for its speakers. This project is motivated by closing this gap for Bengali specifically, through a tokenizer whose design choices are grounded in Bengali orthography and evaluated with a rigorous, multi-metric benchmark rather than a single tokens-per-word figure.

2 Research Problem and Background

The research problem addressed in this project is that existing Bengali tokenizers reduce token fertility to some extent, but no existing approach combines a linguistically grounded segmentation unit with a modeling objective suited to Bengali morphology, and no existing approach isolates which of these two design choices is responsible for observed efficiency gains.

Bengali orthography is organized around aksharas, or orthographic syllables, which may combine a consonant, an optional string of additional consonants joined through a virama (hasanta), and an optional dependent vowel sign. General-purpose byte-level tokenizers, such as those used in GPT-4 and LLaMA, have no representation of this structure and frequently split a single akshara across multiple tokens, producing subword units that do not correspond to any real orthographic or linguistic unit.

Bengali-specific tokenizers exist, but each has limitations relevant to this project. BengaliBPE [2] is grapheme-aware but retains the byte pair encoding merge algorithm, and is evaluated only on downstream text classification rather than on tokenization efficiency itself. TituLLMs [3] reduces tokens-per-word for Bangla through vocabulary extension, but this reduction is a byproduct of building a general-purpose language model rather than a targeted tokenizer contribution. This background motivates a project that treats tokenizer design as the primary object of study, not a side effect of building a larger system.

3 Literature Review and Research Gap

3.1 Cross-Lingual Tokenizer Fairness and Cost

Petrov et al. [1] established that tokenizer design creates measurable and substantial cost disparities between languages, using the FLORES-200 parallel corpus to show token count gaps of up to fifteen times relative to English, with a comparison limited to English as the reference point. The Token Tax [4] finds sixteen African languages facing similar token inflation using the AfriMMLU benchmark, showing that the fairness problem is not exclusive to the language pairs originally studied. Ali et al. [5] show, through a large-scale study training twenty-four tokenizers and corresponding language models, that English-centric tokenizers can increase training cost for other languages by up to sixty-eight percent, and that common intrinsic metrics such as fertility and parity do not always correlate cleanly with downstream model performance, a caution that motivates this project’s use of multiple complementary metrics rather than a single measure.

3.2 Bengali and Bangla-Specific Tokenizer and Language Model Efforts

BengaliBPE [2] is the closest existing work to this project, applying grapheme-level initialization to byte pair encoding for Bengali, but evaluating only macro-F1 on news classification without reporting fertility or cost metrics. TituLLMs [3] extends the LLaMA-3.2 tokenizer vocabulary with Bangla tokens and reports an approximate four-times reduction in sequence length, but this is reported as one outcome among many in a broader model-building effort. TigerLLM [6] trains Bangla language models on curated textbook and instruction data, but continues to use the base LLaMA or Gemma tokenizer, which the paper itself describes as inadequate for Bangla. BanglaBERT [7] introduces the first Bangla language understanding benchmark and a large pretraining corpus. But it does not address tokenizer efficiency directly as it relies on a standard WordPiece vocabulary. A viability study on Bengali language models [8] underscores the practical importance of efficiency gains such as those targeted by this project.

3.3 Grapheme-Level and Script-Aware Segmentation

Velayuthan and Sarveswaran [9] shows that grapheme-cluster-based tokenization can outperform standard byte-level byte pair encoding for Hindi, Tamil, and Sinhala, three scripts having similar structural features with Bengali, including dependent vowel signs and conjunct consonants. A study on rich morphology [10] examines Hindi, Telugu, and English along with a custom gold morpheme-segmentation dataset, that reinforces segmentation unit choice, affects the tokenization quality of morphologically rich languages. These works together provide the justification for using grapheme clusters rather than bytes as the initialization unit for a Bengali tokenizer.

3.4 Morphology-Aware, Unigram, and Alternative Approaches to BPE

A few studies question if byte pair encoding is the right modeling for morphologically rich languages. Bostrom and Durrett [11] propose that byte pair encoding is suboptimal for llm pretraining compared to a unigram model objective. This is because byte pair encoding merges are greedily chosen by frequency. Shahriar and Barbosa [12] test WordPiece, unigram, and character-level tokenization for Bengali and Hindi at BERT scale, finding that the unigram and character-level tokenizers outperform WordPiece for both languages, although the study does not test larger generative models. MorphBPE [13] introduces a morpheme-boundary-aware constraint on byte pair encoding merges and, notably, challenges the common assumption that lower fertility alone indicates a better tokenizer, a point this project addresses by reporting multiple metrics rather than fertility in isolation. Parity-Aware Byte-Pair Encoding [14] modifies the byte pair encoding merge selection process to target the currently worst-compressed language at each step, achieving more equal token counts across languages with little loss in overall compression, and represents a candidate merge strategy that could be adapted for Bengali in future extensions of this work. SuperBPE [15] challenges the whitespace-bounded assumption of standard subword tokenization by learning multi-word "superword" tokens, illustrating more broadly that the unit over which a tokenizer operates, not only the algorithm used to build its vocabulary, is a first-class design decision, which is the same principle this project applies to Bengali through grapheme-cluster initialization.

3.5 Evaluation Metrics Beyond Tokens-Per-Word

Rust et al. [16] show that a tokenizer’s fit to a specific language affects monolingual performance of multilingual models, establishing that tokenizer quality should be measured directly rather than assumed from model performance alone. A more recent study [17] proposes Single Token Retention Rate as a complementary metric to fertility, evaluated across formal text, informal text, and common word lists, and argues that fertility alone can be a misleading indicator of tokenizer quality. A study focused on Indian languages [18] uses the Sangraha corpus together with FLORES-200 and IndicXtreme to characterize tokenization challenges specific to Indian languages, including Bengali. Two further studies extend tokenizer evaluation to individual and multiple Indian languages respectively: a case study on Assamese [19] using a single literary text, and a broader evaluation [20] covering all twenty-two official Indian languages using literary and religious texts. Together, these studies motivate this project’s decision to report fertility, characters-per-token, parity relative to English, and Single Token Retention Rate jointly, rather than relying on any single metric.

3.6 Corpus Resources

IndicLLMSuite [21] introduces the Sangraha corpus, comprising approximately 251 billion tokens across twenty-two Indian languages, of which the Bengali share is approximately 30 billion tokens. This corpus is proposed as the primary training resource for this project, as a cleaner and more systematically curated alternative to general web-crawled corpora such as CC-100.

3.7 Research Gap

The literature reviewed above establishes three relevant strands: cross-lingual tokenizer fairness and cost, Bengali-specific tokenizer and language model efforts, and technical work on grapheme-level segmentation and unigram or morphology-aware modeling objectives. However, these strands have not been combined and separated in a controlled manner for Bengali. No existing study applies grapheme-cluster initialization together with a unigram language model objective specifically for Bengali, and no existing study isolates, through ablation, whether observed efficiency gains come from the segmentation unit, the modeling objective, or their combination. Furthermore, existing Bengali tokenizer evaluations rely mainly on tokens-per-

word or downstream task accuracy, without jointly reporting parity relative to English, Single Token Retention Rate, and characters-per-token, all of which together provide a more complete picture of efficiency and fairness. This project addresses that gap.

4 Research Questions and Hypotheses

- **RQ1.** Does grapheme-cluster initialization reduce token fertility compared to byte-level or character-level initialization, holding the modeling objective constant?
- **RQ2.** Does a unigram language model objective reduce token fertility compared to byte pair encoding, holding the segmentation unit constant?
- **RQ3.** Does the combined grapheme-cluster and unigram language model tokenizer outperform existing Bengali tokenizers and general-purpose tokenizers on fertility, parity relative to English, and Single Token Retention Rate?

5 Proposed Methodology and Justification

The segmentation unit for this project is the akshara, identified using Unicode extended grapheme clusters under Unicode Standard Annex 29, which since Unicode version 15.0 include the Indic conjunct break rule. This rule makes sure that segmentation does not split a linguistically meaningful unit. This choice is justified directly by prior work showing that grapheme-level segmentation outperforms byte-level segmentation for complex, non-Latin scripts [9, 10].

The modeling objective is a unigram language model rather than byte pair encoding, justified by prior findings that the unigram objective aligns more closely with the structure of morphologically rich languages such as Bengali [11, 12].

Because segmentation unit and modeling objective are independent design choices, the methodology follows a two-by-two ablation design: grapheme versus byte-level initialization, crossed with unigram versus byte pair encoding objective. All four variants are trained on the same corpus at the same fixed vocabulary size, so that any difference in outcome can be attributed to the specific design choice under study rather than to an uncontrolled variable such as vocabulary size or training data.

For training at scale, grapheme clusters are remapped to synthetic identifiers and passed into the SentencePiece trainer for both unigram and byte pair encoding modes. This allows the project to retain the linguistic novelty of grapheme-cluster segmentation while using a production-grade and well-tested training implementation rather than a slower custom implementation, which is reserved for small-scale pipeline validation only.

6 Dataset Details

The primary training corpus is the Bengali subset of Sangraha, introduced by IndicLLMSuite [21], comprising approximately 30 billion tokens. This corpus is used as a cleaner alternative to general web-crawled corpora such as CC-100.

The evaluation set is a held-out collection of Bengali text spanning multiple domains, including news, literature, social media, and Wikipedia, to avoid drawing conclusions from a single register or genre. Where appropriate, standard multilingual parallel resources such as FLORES-200, used previously for cross-lingual tokenizer fairness evaluation [1, 18], are incorporated for the parity-relative-to-English metric, since a parallel corpus allows a controlled, meaning-equivalent comparison between Bengali and English token counts.

Baseline tokenizers for comparison are BengaliBPE [2], TituLLMs [3], GPT-4, and LLaMA. Official released tokenizers and vocabularies are used for these baselines where available, rather than reimplementations, in order to preserve external validity.

No human participants are involved in this project. All corpora are drawn from openly released research datasets and are accessed and used according to their respective stated licenses.

7 Implementation Plan

1. Build and validate the akshara segmentation module using Unicode extended grapheme clusters with Indic conjunct break support.
2. Implement the grapheme-to-synthetic-identifier remapping layer required to train grapheme-initialized variants using the SentencePiece trainer.
3. Train all four ablation variants, grapheme or byte-level initialization crossed with unigram or byte pair encoding objective, at small scale first, to confirm the training pipeline is correct.
4. Scale training to the full Sangraha Bengali corpus and fix a common vocabulary size across all four variants.
5. Package each trained tokenizer with a consistent interface so that all four internal variants and the external baselines can be evaluated through the same benchmarking code.

8 Evaluation and Analysis Plan

1. Compute fertility, characters-per-token, parity relative to English, and Single Token Retention Rate for all four internal variants and all external baselines on the held-out multi-domain evaluation set.
2. Report the mean and variance of each metric across evaluation domains, rather than a single pooled score, to reveal whether performance is consistent or domain-dependent.
3. Run paired comparisons per sentence or document across tokenizers, so that content variation is controlled for rather than confounding the comparison between tokenizers.
4. Apply paired statistical testing, using a paired t-test or a Wilcoxon signed-rank test depending on the distribution of the metric differences, to assess whether observed fertility differences between variants are statistically significant.
5. Cross-validate the custom training components against a standard SentencePiece unigram model trained on the same data, to detect implementation errors before drawing conclusions from the ablation.
6. Interpret RQ1 and RQ2 directly from the relevant pairwise comparisons within the two-by-two ablation matrix, and interpret RQ3 from the comparison between the combined grapheme-plus-unigram variant and all external baselines.

9 Task Distribution, Feasibility, and Risks

9.1 Task Distribution

This project is carried out by two members, and responsibilities are split by skill set. This division is not rigid and may adjust as the project progresses.

Shahriar leads the technical implementation: building and validating the akshara segmentation module, setting up the SentencePiece training pipeline, fixing shared hyperparameters such as vocabulary size and maximum span length, training all four ablation variants at small and full scale, implementing the benchmarking metrics, cross-validating the custom trainer against SentencePiece, and fixing implementation bugs.

Raiyan leads the research and writing: finalizing the literature review, sourcing the Sangraha and Indi-cLLMSuite corpora, inspecting tokenizer output for correctness, preparing the held-out evaluation set and documenting data provenance, running experiments and statistical analysis, and drafting the methodology, discussion, and limitations sections.

Both members review the final report together. Raiyan leads the final writing and language polish, and Shahriar verifies technical claims and reproducibility details before submission.

9.2 Feasibility and Risks

The project is feasible within its planned scope. The training corpus, evaluation resources, and baseline tokenizers are all openly available, and the core training method relies on the well established SentencePiece library rather than an unproven custom system.

- **Dataset domain bias.** The training or evaluation corpus may skew toward certain domains. This is mitigated by using a multi domain held out evaluation set.
- **Vocabulary size confound.** Unequal vocabulary size across compared tokenizers could bias results. This is mitigated by fixing vocabulary size across all trained variants.
- **Weak or unfair baselines.** Reimplemented baselines could be unfairly weak. This is mitigated by using officially released baseline tokenizers where possible.
- **Implementation bugs.** Errors in the custom training components could affect results. This is mitigated by cross validating against SentencePiece unigram.
- **Limited generalizability.** Findings may not extend beyond Bengali. This is addressed by scoping all claims explicitly to Bengali and noting cross language generalization as future work.

References

- [1] Aleksandar Petrov, Emanuele La Malfa, Philip Torr, and Adel Bibi. Language model tokenizers introduce unfairness between languages. *Advances in neural information processing systems*, 36:36963–36990, 2023.
- [2] Firoj Ahmmed Patwary and Abdullah Al Noman. Evaluating subword tokenization techniques for bengali: A benchmark study with bengalibpe. *arXiv preprint arXiv:2511.05324*, 2025.
- [3] Shahriar Kabir Nahin, Rabindra Nath Nandi, Sagor Sarker, Quazi Sarwar Muhtaseem, Md Kowsher, Apu Chandraw Shill, Md Ibrahim, Mehadi Hasan Menon, Tareq Al Muntasir, and Firoj Alam. Tit-ullms: A family of bangla llms with comprehensive benchmarking. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24922–24940, 2025.

- [4] Jessica M Lundin, Ada Zhang, Nihal Karim, Hamza Louzan, Guohao Wei, David Ifeoluwa Adelani, and Cody Carroll. The token tax: Systematic bias in multilingual tokenization. In *Proceedings of the 7th Workshop on African Natural Language Processing (AfricaNLP 2026)*, pages 103–112, 2026.
- [5] Mehdi Ali, Michael Fromm, Klaudia Thellmann, Richard Rutmann, Max Lübbering, Johannes Levelling, Katrin Klug, Jan Ebert, Niclas Doll, Jasper Schulze Buschhoff, et al. Tokenizer choice for llm training: Negligible or crucial? *arXiv preprint arXiv:2310.08754*, 2023.
- [6] Nishat Raihan and Marcos Zampieri. Tigerllm-a family of bangla large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 887–896, 2025.
- [7] Abhik Bhattacharjee, Tahmid Hasan, Wasi Ahmad, Kazi Samin Mubasshir, Md Saiful Islam, Anindya Iqbal, M Sohel Rahman, and Rifat Shahriyar. Banglabert: Language model pretraining and benchmarks for low-resource language understanding evaluation in bangla. In *Findings of the Association for Computational Linguistics: NAACL 2022*, pages 1318–1327, 2022.
- [8] Tamzeed Mahfuz, Satak Kumar Dey, Ruwad Naswan, Hasnaen Adil, Khondker Salman Sayeed, and Haz Sameen Shahgir. Too late to train, too early to use? a study on necessity and viability of low-resource bengali llms. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 1183–1200, 2025.
- [9] Menan Velayuthan and Kengatharaiyer Sarveswaran. Egalitarian language representation in language models: It all begins with tokenizers. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5987–5996, 2025.
- [10] Saketh Reddy Vemula, Sandipan Dandapat, Dipti Misra Sharma, and Parameswari Krishnamurthy. Rethinking tokenization for rich morphology: The dominance of unigram over bpe and morphological alignment. In *The 14th International Joint Conference on Natural Language Processing and The 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 232–252, 2025.
- [11] Kaj Bostrom and Greg Durrett. Byte pair encoding is suboptimal for language model pretraining. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 4617–4624, 2020.
- [12] Arif Shahriar and Denilson Barbosa. Improving bengali and hindi large language models. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 8719–8731, 2024.
- [13] Ehsaneddin Asgari, Yassine El Kheir, and Mohammad Ali Sadraei Javaheri. Morphbpe: A morpho-aware tokenizer bridging linguistic complexity for efficient llm training across morphologies. *arXiv preprint arXiv:2502.00894*, 2025.
- [14] Negar Foroutan, Clara Meister, Debjit Paul, Joel Niklaus, Sina Ahmadi, Antoine Bosselut, and Rico Sennrich. Parity-aware byte-pair encoding: Improving cross-lingual fairness in tokenization. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7514–7538, 2026.
- [15] Alisa Liu, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A Smith, and Yejin Choi. Superbpe: Space travel for language models. *arXiv preprint arXiv:2503.13423*, 2025.

- [16] Phillip Rust, Jonas Pfeiffer, Ivan Vulić, Sebastian Ruder, and Iryna Gurevych. How good is your tokenizer? on the monolingual performance of multilingual language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 3118–3135, 2021.
- [17] Mir Tafseer Nayeem, Sawsan Alqahtani, Md Tahmid Rahman Laskar, Tasnim Mohiuddin, and M Saiful Bari. Beyond fertility: Analyzing strr as a metric for multilingual tokenization evaluation. *arXiv preprint arXiv:2510.09947*, 2025.
- [18] Maharaj Brahma, NJ Karthika, Rajat Verma, Nagasai Saketh Naidu, Rohit Saluja, Maunendra Sankar Desarkar, and Ganesh Ramakrishnan. Multilingual tokenization through the lens of indian languages: Challenges and insights. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 32614–32632, 2026.
- [19] Sagar Tamang and Dibya Jyoti Bora. Performance evaluation of tokenizers in large language models for the assamese language. *International Journal of Information Technology*, 17(4):2329–2332, 2025.
- [20] Sagar Tamang and Dibya Jyoti Bora. Evaluating tokenizer performance of large language models across official indian languages. *arXiv preprint arXiv:2411.12240*, 2024.
- [21] Mohammed Safi Ur Rahman Khan, Priyam Mehta, Ananth Sankar, Umashankar Kumaravelan, Sumanth Doddapaneni, Sparsh Jain, Anoop Kunchukuttan, Pratyush Kumar, Raj Dabre, Mitesh M Khapra, et al. Indicllmsuite: A blueprint for creating pre-training and fine-tuning datasets for indian languages. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15831–15879, 2024.